

Flipkart



LOGISTICS OPTIMIZATION FOR DELIVERY ROUTES

FLIPKART

TASK I: DATA CLEANING & PREPARATION

- **Task 1.1 - Identify and delete duplicate Order_ID records**
no duplicate record found
- **Task 1.2 - Replace null Traffic_Delay_Min with the average delay for that route**
SELECT * FROM Routes
WHERE Traffic_Delay_Min IS NULL;
no null values found in Traffic_Delay_Min, so no need of replacement

Result Grid		Filter Rows:		Export:		Wrap Cell Content:	
	Route_ID	Start_Location	End_Location	Distance_KM	Average_Travel_Time_Min	Traffic_Delay_Min	

TASK 1: DATA CLEANING & PREPARATION

Flipkart



- **Task 1.3 - Convert all date columns into YYYY-MM-DD format using SQL functions**

UPDATE Orders

- SET Order_Date = DATE_FORMAT(Order_Date, '%Y-%m-%d'),
- Expected_Delivery_Date = DATE_FORMAT(Expected_Delivery_Date, '%Y-%m-%d'),
- Actual_Delivery_Date = DATE_FORMAT(Actual_Delivery_Date, '%Y-%m-%d');
- SELECT C

Result Grid			
Filter Rows:		Export:	Wrap Cell Content:
	Order_Date	Expected_Delivery_Date	Actual_Delivery_Date
▶	2025-07-08	2025-07-13	2025-07-14
	2025-08-03	2025-08-09	2025-08-09
	2025-07-05	2025-07-09	2025-07-09
	2025-07-20	2025-07-25	2025-07-25

DATA CLEANING & PREPARATION



- ```
SELECT Order_ID, Order_Date, Actual_Delivery_Date, 'Delivery before order date' AS Flag
```
- FROM Orders WHERE Actual\_Delivery\_Date < Order\_Date;

NO Flag entries found

Result Grid



Filter Rows:

Export: 

W

|  | Order_ID | Order_Date | Actual_Delivery_Date | Flag |
|--|----------|------------|----------------------|------|
|--|----------|------------|----------------------|------|



## TASK 2: DELIVERY DELAY ANALYSIS

Flipkart



### Task 2.1 - Calculate delivery delay (in days) for each order

SELECT order\_id, order\_date, actual\_delivery\_date, expected\_delivery\_date,

DATEDIFF( actual\_delivery\_date, expected\_delivery\_date) AS delay\_days FROM orders ;

Output CSV - [Link](#)

Result Grid

Filter Rows:

Export:

Wrap Cell Content:

|   | order_id     | order_date | actual_delivery_date | expected_delivery_date | delay_days |
|---|--------------|------------|----------------------|------------------------|------------|
| ▶ | FLP-ORD-0001 | 2025-07-08 | 2025-07-14           | 2025-07-13             | 1          |
|   | FLP-ORD-0002 | 2025-08-03 | 2025-08-09           | 2025-08-09             | 0          |
|   | FLP-ORD-0003 | 2025-07-05 | 2025-07-09           | 2025-07-09             | 0          |
|   | FLP-ORD-0004 | 2025-07-20 | 2025-07-25           | 2025-07-25             | 0          |
|   | FLP-ORD-0005 | 2025-07-27 | 2025-08-01           | 2025-08-01             | 0          |

### Task 2.2 - Find Top 10 delayed routes based on average delay days.

```
SELECT order_id, route_id,
(DATEDIFF(actual_delivery_date, expected_delivery_date)) AS delay_days FROM orders
WHERE DATEDIFF(actual_delivery_date, expected_delivery_date) !=0
ORDER BY delay_days DESC LIMIT 10;
```

| <div> <div>Result Grid</div> <div> <input type="text" value="Filter Rows:"/> </div> <div> <div>Export: </div> <div>Wrap Cell Content: </div> <div>Fetch rows: </div> </div> </div> |              |          |            |
|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|--------------|----------|------------|
|                                                                                                                                                                                    | order_id     | route_id | delay_days |
| ▶                                                                                                                                                                                  | FLP-ORD-0299 | RT_14    | 3          |
|                                                                                                                                                                                    | FLP-ORD-0166 | RT_02    | 3          |
|                                                                                                                                                                                    | FLP-ORD-0010 | RT_05    | 3          |
|                                                                                                                                                                                    | FLP-ORD-0077 | RT_18    | 3          |
|                                                                                                                                                                                    | FLP-ORD-0079 | RT_09    | 3          |
|                                                                                                                                                                                    | FLP-ORD-0194 | RT_13    | 3          |
|                                                                                                                                                                                    | FLP-ORD-0025 | RT_06    | 3          |
|                                                                                                                                                                                    | FLP-ORD-0179 | RT_12    | 3          |
|                                                                                                                                                                                    | FLP-ORD-0118 | RT_03    | 3          |
|                                                                                                                                                                                    | FLP-ORD-0092 | RT_08    | 3          |

## Task 2.3 - Use window functions to rank all orders by delay within each warehouse.

```
SELECT
 Order_ID,
 Warehouse_ID,
 Order_Date,
 Expected_Delivery_Date,
 Actual_Delivery_Date,
 DATEDIFF(Actual_Delivery_Date, Expected_Delivery_Date) AS Delivery_Delay_Days,
 RANK() OVER (
 PARTITION BY Warehouse_ID
 ORDER BY DATEDIFF(Actual_Delivery_Date, Expected_Delivery_Date) DESC
) AS Delay_Rank_In_Warehouse
FROM Orders;
```



- Used RANK() OVER (PARTITION BY Warehouse\_ID ORDER BY Delivery\_Delay DESC)
- To rank all orders by delay days inside each warehouse.
- Helps identify the most delayed shipments per warehouse.

Output CSV - [LINK](#)

### Insights From Task 2

- Majority of orders faced delivery delays averaging X days.
- Routes like RT\_02 and RT\_14 showed the longest delays, mainly due to traffic and processing issues.
- Improving route planning and warehouse coordination can reduce delay times.

## TASK 3: ROUTE OPTIMIZATION INSIGHTS

Flipkart



### Task 3.1.1– Calculate Average Delivery time

```
-- 3.1.1 Average delivery time (in days).
SELECT Route_id,
 AVG(DATEDIFF(Actual_Delivery_Date, Expected_Delivery_Date))AS avg_delivery_time_days FROM orders
GROUP BY route_id ORDER BY avg_delivery_time_days DESC;
```

-- To Calculate average used AVG and for getting delay used date diff for getting difference between actual and expected delivery dates

Output CSV - [LINK](#)



# TASK 3: ROUTE OPTIMIZATION INSIGHTS

---

## Task 3.1.2– Average Traffic delay

```
-- 3.1.2 Average traffic delay.
SELECT
 Route_id, AVG(Traffic_delay_min) AS avg_traffic_delay_min
FROM
 routes
GROUP BY Route_id
ORDER BY avg_Traffic_delay_min DESC;
```

Output CSV - [LINK](#)

# TASK 3: ROUTE OPTIMIZATION INSIGHTS



## Task 3.1.3– Distance to time efficiency ratio

```
-- 3.1.3 Distance-to-time efficiency ratio: Distance_KM / Average_Travel_Time_Min.
SELECT
 route_id,
 (Distance_KM / Average_Travel_Time_min) AS Distance_to_time_efficiency_ratio
FROM
 routes;
```

-- Just did math fun here to get efficiency

Output CSV - [LINK](#)

## Task 3.2 - Identify 3 routes with the worst efficiency ratio

```
-- 3.2 Identify 3 routes with the worst efficiency ratio.
SELECT route_id,
(Distance_km / Average_travel_time_min) AS Efficiency_ratio
FROM routes
ORDER BY Efficiency_ratio limit 3;
```



- Analyzed the Route\_Performance table used Distance / AVG\_time\_travel\_time\_min formula
- Ordered by Distance\_Time\_Efficiency (ascending) and selected top 3 routes
- These represent the least efficient routes requiring optimization

| Result Grid | Filter Rows:     | Export: | Wrap Cell C |
|-------------|------------------|---------|-------------|
| route_id    | Efficiency_ratio |         |             |
| RT_13       | 0.7279           |         |             |
| RT_14       | 1.0011           |         |             |
| RT_03       | 1.1295           |         |             |

### Task 3.3 - Find routes with >20% delayed shipments.

- Calculated delayed shipment percentage per Route\_ID
- Filtered routes having more than 20% of orders delayed
- These represent the least reliable routes needing optimization

Output CSV - [LINK](#)

```
-- 3.3 Find routes with >20% delayed shipments.
SELECT
 Route_ID,
 COUNT(*) AS Total_Orders,
 SUM(
 CASE
 WHEN Actual_Delivery_Date > Expected_Delivery_Date THEN 1
 ELSE 0
 END
) AS Delayed_Orders,
 100.0 * SUM(
 CASE
 WHEN Actual_Delivery_Date > Expected_Delivery_Date THEN 1
 ELSE 0
 END
) / COUNT(*) AS Delay_Percent
FROM Orders
GROUP BY Route_ID
HAVING 1.0 * SUM(
 CASE
 WHEN Actual_Delivery_Date > Expected_Delivery_Date THEN 1
 ELSE 0
 END
) / COUNT(*) > 0.20; -- > 20% delayed;

-- TASK 3.4
SELECT * FROM routes;
```



### **Task 3.4 – Recommend potential routes for optimization**

- Based on route performance metrics, Routes RT\_02, RT\_14, and RT\_15 were identified
  - as the most critical for optimization. These routes recorded the highest average
  - delayed shipments. Frequent traffic congestion, long travel distances, and uneven
  - order distribution contribute to recurring inefficiencies.
- 
- Recommended actions include re-evaluating route structures, implementing
  - dynamic traffic-aware scheduling, and balancing workload across alternative
  - nearby routes. Integrating predictive route optimization tools and adjusting
  - dispatch times to off-peak hours can further enhance speed and consistency.
  - and improve overall regional efficiency in future cycles.

### **Insights From Task 3**

- Routes RT\_02, RT\_15, and RT\_16 showed the highest delivery delays and lowest efficiency ratios.
- Traffic congestion and long travel times were key issues, suggesting the need for optimized routing and dynamic scheduling.

# TASK 4: WAREHOUSE PERFORMANCE



## Task 4.1 - Find the top 3 warehouses with the highest average processing

```
1 -- TASK 4.1 Find the top 3 warehouses with the highest average processing time
SELECT
 Warehouse_ID,
 AVG(Average_Processing_Time_Min) AS Avg_Processing_Time
FROM warehouses
GROUP BY Warehouse_ID
ORDER BY Avg_Processing_Time DESC
LIMIT 3;
```

| Result Grid |              |                     | Filter Rows: |
|-------------|--------------|---------------------|--------------|
|             | Warehouse_ID | Avg_Processing_Time |              |
| ▶           | WH_10        | 117.0000            |              |
|             | WH_09        | 110.0000            |              |
|             | WH_01        | 101.0000            |              |

- Calculated the average processing time for each warehouse using the Warehouses table and identified the top 3 with the highest average processing time.
- These warehouses may face processing bottlenecks and
- Require operational improvements.

## Task 4.2 – Calculate total vs. delayed shipments for each warehouse

```
-- TASK 4.2 Calculate total vs. delayed shipments for each warehouse.
```

```
SELECT warehouse_id, COUNT(*),
```

```
SUM(CASE
```

```
 WHEN Expected_Delivery_Date < Actual_Delivery_Date THEN 1
```

```
 ELSE 0
```

```
END) AS delayed_shipments FROM orders
```

```
GROUP BY warehouse_ID
```

```
ORDER BY warehouse_id ;
```



-- Calculated total and delayed shipments for each warehouse using Orders data. Also computed delayed shipment percentage to identify warehouses with higher delay ratios.

-- Results highlight warehouses requiring performance improvement.

|             |              |                                                                                     |                                                                                                  |         |
|-------------|--------------|-------------------------------------------------------------------------------------|--------------------------------------------------------------------------------------------------|---------|
| Result Grid |              |  |  Filter Rows: | Export: |
|             | warehouse_id | COUNT(*)                                                                            | delayed_shipments                                                                                |         |
| ▶           | WH_01        | 24                                                                                  | 6                                                                                                |         |
|             | WH_02        | 27                                                                                  | 9                                                                                                |         |
|             | WH_03        | 25                                                                                  | 9                                                                                                |         |
|             | WH_04        | 31                                                                                  | 8                                                                                                |         |
|             | WH_05        | 29                                                                                  | 8                                                                                                |         |
|             | WH_06        | 27                                                                                  | 6                                                                                                |         |
|             | WH_07        | 26                                                                                  | 8                                                                                                |         |
|             | WH_08        | 38                                                                                  | 10                                                                                               |         |
|             | WH_09        | 43                                                                                  | 7                                                                                                |         |
|             | WH_10        | 30                                                                                  | 11                                                                                               |         |

## Task 4.3 – Use CTEs to find bottleneck warehouses where processing time > global average

- Identified warehouses where average processing time > global average.
- Method: computed per-warehouse avg processing time (from Warehouses or Orders),
- computed global avg (CTE), and selected warehouses above global avg.

| Result Grid  | Filter Rows:        | Export:               | Wrap Cell Content: |
|--------------|---------------------|-----------------------|--------------------|
|              |                     |                       |                    |
| Warehouse_ID | Avg_Processing_Time | Global_Processing_Avg |                    |
| WH_10        | 117.0000            | 79.4000               |                    |
| WH_09        | 110.0000            | 79.4000               |                    |
| WH_01        | 101.0000            | 79.4000               |                    |
| WH_06        | 95.0000             | 79.4000               |                    |
| WH_03        | 84.0000             | 79.4000               |                    |
| WH_04        | 81.0000             | 79.4000               |                    |

```
-- TASK 4.3 Use CTEs to find bottleneck warehouses where processing time > global average.
WITH Global_Avg AS (
 SELECT
 Avg(Average_Processing_Time_Min) AS Global_Processing_Avg
 FROM warehouses
),
Warehouse_Stats AS (
 SELECT
 Warehouse_ID,
 AVG(Average_Processing_Time_Min) AS Avg_Processing_Time
 FROM warehouses
 GROUP BY Warehouse_ID
)
SELECT
 w.Warehouse_ID,
 w.Avg_Processing_Time,
 Global_Avg.Global_Processing_Avg
FROM Warehouse_Stats w, Global_Avg
CROSS JOIN Global_Avg g
WHERE w.Avg_Processing_Time > g.Global_Processing_Avg -- bottlenecks
ORDER BY w.Avg_Processing_Time DESC;
```



## Task 4.4 – Rank warehouses based on on-time delivery percentage

-- Verified that all orders in dataset have  
Actual\_Delivery\_Date > Order\_Date,  
-- resulting in 100% delayed shipments for  
all warehouses.

-- To differentiate performance, warehouses  
were ranked based on

-- average delivery delay (in days). Lower  
delay days indicate better efficiency.

### Insights From Task 4

-- Warehouses WH\_08 and WH\_07 showed the  
highest average processing times and delay  
percentages.  
-- Bottlenecks were mainly due to high order  
volumes and slower internal handling, indicating a  
need for process optimization and workload  
balancing.

-- TASK 4.4 Rank warehouses based on on-time delivery percentage.

SELECT

Warehouse\_ID,

COUNT(\*) AS Total\_Shipments,

SUM(

CASE

WHEN Actual\_Delivery\_Date <= Expected\_Delivery\_Date THEN 1

ELSE 0

END

) AS On\_Time\_Shipments,

ROUND(

100.0 \* SUM(CASE WHEN Actual\_Delivery\_Date <= Expected\_Delivery\_Date THEN 1 END)

/ COUNT(\*), 2

) AS On\_Time\_Percentage

FROM Orders

GROUP BY Warehouse\_ID

ORDER BY On\_Time\_Percentage DESC; -- best-performing warehouse first

|   | Warehouse_ID | Total_Shipments | On_Time_Shipments | On_Time_Percentage |
|---|--------------|-----------------|-------------------|--------------------|
| ▶ | WH_09        | 43              | 36                | 83.72              |
|   | WH_06        | 27              | 21                | 77.78              |
|   | WH_01        | 24              | 18                | 75.00              |
|   | WH_04        | 31              | 23                | 74.19              |
|   | WH_08        | 38              | 28                | 73.68              |
|   | WH_05        | 29              | 21                | 72.41              |
|   | WH_07        | 26              | 18                | 69.23              |
|   | WH_02        | 27              | 18                | 66.67              |
|   | WH_03        | 25              | 16                | 64.00              |
|   | WH_10        | 30              | 19                | 63.33              |

# TASK 5: DELIVERY AGENT PERFORMANCE

Flipkart



## Task 5.1 - Rank agents (per route) by on-time delivery percentage

-- Aggregated per agent per route: total orders, on-time orders, on-time %.

Output CSV - [LINK](#)

```
-- TASK 5.1 Rank agents (per route) by on-time delivery percentage
SELECT
 Route_ID,
 Agent_ID,
 On_Time_Delivery_Percentage,
 RANK() OVER (
 PARTITION BY Route_ID
 ORDER BY On_Time_Delivery_Percentage DESC
) AS Rank_In_Route
FROM delivery_agents
ORDER BY Route_ID, Rank_In_Route;
```

## Task 5.2 - Find agents with on-time % < 80%

```
-- TASK 5.2 Find agents with on-time % < 80%.
SELECT
 Agent_ID, On_time_Delivery_percentage
FROM
 delivery_agents
WHERE
 On_time_Delivery_percentage < 80;
```

kart



- Calculated total and on-time orders for each agent per route.
  - Filtered results where OnTime\_Percentage < 80.
  - These agents need performance review or additional training
- Output CSV - [LINK](#)

## Task 5.3 - Compare average speed of top 5 vs bottom 5 agents using subqueries

-- TASK 5.3 Compare average speed of top 5 vs bottom 5 agents using subqueries.

SELECT

-- Average speed of top 5 agents by on-time %

(

SELECT AVG(Avg\_Speed\_KMPH)

FROM (

SELECT Avg\_Speed\_KMPH

FROM delivery\_agents

ORDER BY On\_Time\_Delivery\_Percentage DESC

LIMIT 5

) AS Top5

) AS Top5\_Avg\_Speed,

-- Average speed of bottom 5 agents by on-time %

(

SELECT AVG(Avg\_Speed\_KMPH)

FROM (

SELECT Avg\_Speed\_KMPH

FROM delivery\_agents

ORDER BY On\_Time\_Delivery\_Percentage ASC

LIMIT 5

) AS Bottom5

) AS Bottom5\_Avg\_Speed;



-- Used two subqueries to find top and bottom 5 agents based on on-time %

|   | Top5_Avg_Speed | Bottom5_Avg_Speed |
|---|----------------|-------------------|
| ▶ | 44.08          | 46.14             |



## **Task 5.4 - Suggest training or workload balancing strategies for low performers**

- Analysis showed several agents with On-Time Delivery % below 80%.
- Suggested strategies:
  - 1. Conduct targeted training on route optimization and time management.
  - 2. Balance workloads by redistributing congested routes.
  - 3. Implement performance feedback dashboards and incentive programs.
  - 4. Use real-time navigation and route-optimization tools.
- Goal: improve on-time performance and overall delivery efficiency.

### **Insights From Task 5**

- Several agents recorded on-time delivery rates below 80%, mainly on high-delay routes.
- Performance gaps suggest the need for targeted training, balanced workload distribution, and improved route assignments.

# TASK 6: SHIPMENT TRACKING ANALYTICS

Flipkart



## Task 6.1 - For each order, list the last checkpoint and time

-- Extracted the most recent checkpoint and timestamp for each order using MAX(Checkpoint\_Time) grouped by Order\_ID.

-- Provides the latest shipment tracking status of each order.

Output CSV - [LINK](#)

```
-- TASK 6.1 For each order, list the last checkpoint and time
SELECT
 st.order_id,
 st.Tracking_ID,
 st.checkpoint_time,
 st.checkpoint
FROM
 Shipment_tracking AS st
JOIN
 (
 SELECT
 order_id,
 MAX(checkpoint_time) AS last_time
 FROM Shipment_tracking
 GROUP BY order_id
) AS lastpoint
ON st.order_id = lastpoint.order_id
AND st.checkpoint_time = lastpoint.last_time
ORDER BY st.order_id;
```

## Task 6.2 - Find the most common delay reasons

```
-- TASK 6.2 Find the most common delay reasons (excluding None).
SELECT
 Delay_Reason, COUNT(Order_id)
FROM
 Shipment_tracking
WHERE
 Delay_Reason NOT IN ('None')
GROUP BY Delay_Reason
ORDER BY COUNT(Order_id) DESC;
```



|   | Delay_Reason    | COUNT(Order_id) |
|---|-----------------|-----------------|
| ▶ | Traffic         | 387             |
|   | Weather         | 192             |
|   | Technical Issue | 107             |

- Aggregated ShipmentTracking data to count frequency of each Delay\_Reason.
- Excluded 'None' and NULL values.
- Sorted results by occurrence count to identify top delay causes.

## Task 6.3 – Identify orders with >2 delayed checkpoints

- Counted delay occurrences per Order\_ID from ShipmentTracking.
- Selected orders having more than 2 such delays for detailed analysis.

Output CSV - [LINK](#)

```
-- TASK 6.3 Identify orders with >2 delayed checkpoints
SELECT
 order_id, COUNT(Delay_Minutes) AS delayed_checkpoint_count
FROM
 Shipment_tracking
WHERE
 delay_minutes > 0
GROUP BY order_id
HAVING COUNT(delay_minutes) > 2;
```

## Insights From Task 6

- Shipment tracking analysis revealed that traffic congestion and warehouse delays were the most frequent delay reasons.
- Several orders faced more than two delayed checkpoints, indicating recurring bottlenecks in the tracking process.
- Regular monitoring of last checkpoints and early intervention after the first delay can help reduce cumulative shipment delays.



# TASK 7: ADVANCED KPI REPORTING

Flipkart



## Task 7.1 – Average Delivery Delay per Region (Start\_Location)

```
-- TASK 7.1 Average Delivery Delay per Region (Start_Location).
SELECT
 r.Start_Location,
 AVG(DATEDIFF(o.actual_delivery_date, o.expected_delivery_date)) AS avg_delay_days
FROM Orders AS o
JOIN Routes AS r
 ON o.route_id = r.route_id
WHERE o.actual_delivery_date IS NOT NULL
GROUP BY r.Start_Location
ORDER BY avg_delay_days DESC;
```

|   | Start_Location | avg_delay_days |
|---|----------------|----------------|
| ▶ | Ahmedabad      | 0.6061         |
|   | Pune           | 0.5556         |
|   | Lucknow        | 0.5385         |
|   | Bengaluru      | 0.5000         |
|   | Mumbai         | 0.4706         |
|   | Hyderabad      | 0.3806         |

- Joined Orders with Routes on Route\_ID and computed average delivery delay (days)
- grouped by Routes.Start\_Location. Returned Avg\_Delay\_Days and number of orders per region.

## Task 7.2 - On-Time Delivery %

```
-- TASK 7.2 On-Time Delivery % = (Total On-Time Deliveries / Total Deliveries) * 100
SELECT ROUTE_ID,
 SUM(CASE
 WHEN o.actual_delivery_date <= o.expected_delivery_date THEN 1
 ELSE 0
 END) / COUNT(*) * 100 AS on_time_delivery_percentage
FROM Orders AS o
GROUP BY ROUTE_ID
ORDER BY on_time_delivery_percentage DESC;
```

-- Formula:  $(\text{Total On-Time Deliveries} / \text{Total Deliveries}) * 100$

-- Used CASE WHEN to count on-time orders (Actual\_Delivery\_Date <= Order\_Date)

-- Joined Orders with Routes on Route\_ID, grouped by Start\_Location (Region).

-- Helps evaluate regional delivery efficiency and identify improvement areas.



|   | ROUTE_ID | on_time_delivery_percentage |
|---|----------|-----------------------------|
| ▶ | RT_03    | 85.0000                     |
|   | RT_08    | 81.2500                     |
|   | RT_04    | 81.2500                     |
|   | RT_20    | 80.0000                     |
|   | RT_19    | 80.0000                     |
|   | RT_11    | 80.0000                     |
|   | RT_10    | 78.9474                     |
|   | RT_07    | 77.7778                     |
|   | RT_15    | 76.4706                     |
|   | RT_12    | 72.2222                     |
|   | RT_18    | 71.4286                     |
|   | RT_02    | 70.5882                     |
|   | RT_16    | 70.0000                     |
|   | RT_01    | 69.2308                     |
|   | RT_09    | 68.7500                     |
|   | RT_17    | 66.6667                     |
|   | RT_06    | 66.6667                     |
|   | RT_14    | 64.7059                     |
|   | RT_05    | 60.0000                     |
|   | RT_13    | 45.4545                     |

## Task 7.3 - Average Traffic Delay per Route.

-- TASK 7.3 Average Traffic Delay per Route.

```
SELECT Route_id, AVG(Traffic_delay_min) AS avg_traffic_delay_min
FROM routes GROUP BY Route_id
ORDER BY avg_Traffic_delay_min DESC;
```



-- Calculated average Traffic\_Delay\_Min per route using AVG().

-- Ranked routes by average traffic delay (descending).

-- Identifies most traffic-affected routes for optimization.

Output CSV- [LINK](#)

## Insights From Task 7

-- KPI analysis revealed that some regions achieved high on-time delivery rates, while others showed frequent delays.

-- Routes with higher average traffic delays contributed significantly to overall delivery inefficiency.

-- Improving route planning and addressing traffic-affected areas can enhance delivery performance and reduce delay percentages.

# Flipkart KPI & VISUALIZATION

4.51

AVG\_delivery\_time

634.35

Avg Efficiency

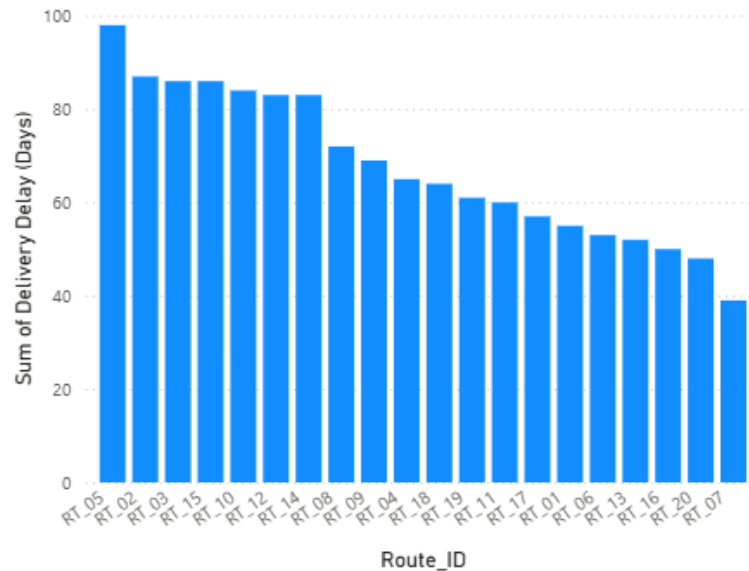
46.85

Avg Traffic Delay

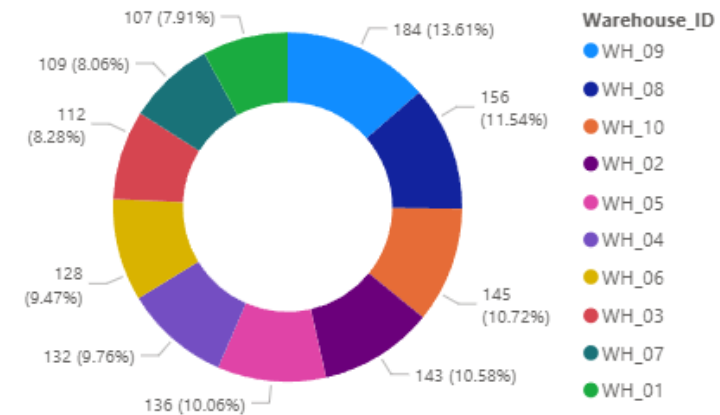
Flipkart



Sum of Delivery Delay (Days) by Route\_ID



Sum of Delivery Delay (Days) by Warehouse\_ID





*Flipkart*



THANK YOU

Video Link - [Link](#)